

Module 6: Lesson 1

Introduction to autoencoders



Outline

- ▶ Introduction to autoencoders
- ▶ Autoencoders as neural networks
- ▶ Linear autoencoders as principal component analysis (PCA)

Autoencoders

Autoencoders are a type of unsupervised ML method that is designed as a reconstruction problem.

Training autoencoders involves the minimization error from reconstructing its inputs $\mathbf{x}$ in two steps

$$\underbrace{\mathbf{x} \longrightarrow f(\mathbf{x}) \longrightarrow \mathbf{z}}_{\text{Encoding}} \underbrace{\longrightarrow g(\mathbf{z}) \longrightarrow \hat{\mathbf{x}}}_{\text{Decoding}}$$

The encoding function f transforms the input $\mathbf{x}$ into encodings $\mathbf{z}$ (latent representations of the data).

The decoding function g transforms the encodings $\mathbf{z}$ back to a reconstruction of the input $\hat{\mathbf{x}}$.

At first, the problem of reconstructing an input may be trivial. But building an autoencoder model may be useful for several tasks, such as:

- ▶ Dimensionality reduction.
- ▶ Denoising.
- ▶ Pre-processing inputs before classification (“whitening”).
- ▶ Outlier detection.

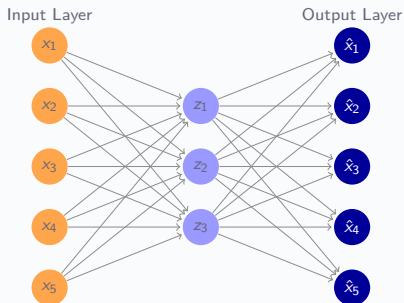
Autoencoders as neural networks

Autoencoders can be represented as a neural network with one or many fully connected hidden layers.

- ▶ The output layer is identical to the input layer, $\mathbf{x}$.
- ▶ Training the neural network implies finding an identity mapping between inputs and outputs.

The hidden activations are called encodings, latent features, or “factors,” $\mathbf{z}$.

- ▶ The encodings capture the dependencies between the different observed variables included in $\mathbf{x}$.



Autoencoders as neural networks

Suppose that we want to encode our feature vector $\mathbf{x}$, with dimension $n \times 1$, into a $k \times 1$ vector of encodings $\mathbf{z}$.

Suppose also that we have p observations of $\mathbf{x}$, say returns of n different stocks over p periods, or information of n features from p individuals.

We stack the information in an $n \times p$ matrix X .

We can represent a single-layer autoencoder as follows, where f and g are activation functions:

$$X \longrightarrow f\left(W_1 X + \mathbf{b}_1 \iota_p^T\right) \longrightarrow Z \longrightarrow g\left(W_2 Z + \mathbf{b}_2 \iota_p^T\right) \longrightarrow \hat{X}$$

where:

- ▶ Encoding weights: W_1 , $k \times n$ matrix. Decoding weights: W_2 , $n \times k$ matrix.
- ▶ Bias terms: $\mathbf{b}_1$, $k \times 1$ vector, and $\mathbf{b}_2$, $n \times 1$ vector.
- ▶ Encodings: Z , $k \times p$ matrix.
- ▶ ι_p , $p \times 1$ vector of ones.
- ▶ $\hat{X}$, $n \times p$ reconstructed input.

Autoencoders as neural networks

Given a choice of activation functions, f and g , and encoding dimension, k , we can train the autoencoder to minimize the reconstruction error computed, say, with the MSE:

$$\frac{1}{np} \|X - \hat{X}\|_F^2$$

where F denotes the Frobenius norm.

As in any other neural network, we can add regularization terms that penalize the weights and the encodings.

Depending on the number of encodings k , we can differentiate among:

- ▶ Undercomplete autoencoders ($n > k$). They gather the most salient features of the data by learning small representations (a “bottleneck”).
- ▶ Overcomplete autoencoders ($n \leq k$). They necessarily need regularization to avoid learning an identity transformation.

We can also increase the depth of the network with additional hidden layers on both sides of the encoding layer.

Linear autoencoders

Suppose you have an autoencoder model with one hidden layer, whose activation functions f and g are linear. The autoencoder can be represented as:

$$X \longrightarrow W_1 X + \mathbf{b}_1 \iota_p^T \longrightarrow Z \longrightarrow W_2 Z + \mathbf{b}_2 \iota_p^T \longrightarrow \hat{X}$$

Then, assuming an MSE loss, training this autoencoder involves the choice of weights and biases that solve:

$$\min_{W_1, W_2, b_1, b_2} \left\| X - W_2 \left(W_1 X + \mathbf{b}_1 \iota_p^T \right) - \mathbf{b}_2 \iota_p^T \right\|_F^2$$

which can be reduced to choosing W_2 to minimize $\left\| X_0 - W_2 (W_2^T W_2)^{-1} W_2^T X_0 \right\|_F^2$.

- ▶ $X_0 = X - X \iota_p / p$ is the matrix of centered observations, i.e., X minus the mean of each feature.
- ▶ W_2 is a solution to the previous problem if its columns form a basis that spans the same subspace as the principal eigenvectors of the variance-covariance matrix of X .
- ▶ Training a single-layer linear autoencoder is roughly equivalent to performing principal component analysis on X .

Principal Components Analysis (PCA)

Principal component analysis consists of a linear transformation of the data to a new coordinate system that explains its variation.

PCA finds a low-dimensional (linear) representation of a data set that contains the maximum level of variation.

For instance, the first principal component of our data X is the linear combination of the uncentered features

$$Z_1 = w_1^T X_0$$

which is the encoding with the largest variance, i.e., $w_1^T X_0 X_0^T w_1$ is maximized.

$w_1 = (w_{11}, w_{21}, \dots, w_{n1})^T$ is a normalized $n \times 1$ vector, such that $\sum_i^n w_{i1}^2 = 1$.

- ▶ The coefficients in w_1 are the *loadings* of the first principal component.
- ▶ We normalize the loadings, otherwise setting these elements to be arbitrarily large could result in an arbitrarily large variance.

PCA: Second and further principal components

The second principal component is the linear combination of X_0 with maximal variance out of all linear combinations that are uncorrelated with Z_1 .

Constraining Z_2 to be uncorrelated with Z_1 is equivalent to constraining the direction w_2 to be orthogonal (perpendicular) to the direction w_1 , $w_1^T w_2 = 0$.

- We also normalize the loadings to have unit norm $\sum_i^n w_{i2}^2 = 1$.

We can proceed in the same manner until reaching the n th principal component, such that the elements $(Z_1, \dots, Z_k)$ explain all the variance of X .

The scores and loadings from PCA are unique up to a sign flip: the variance of Z is the same as that of $-Z$.

PCA and linear autoencoders

Let Σ denote the variance-covariance matrix of X , $\Sigma = \frac{X_0^T X_0}{p-1}$. We can perform the following decomposition:

$$\Sigma = P \Lambda P^T$$

Λ is the diagonal matrix of eigenvalues, sorted from highest to lowest, and P is a matrix whose columns are orthonormal eigenvectors.

- ▶ The eigenvectors are called the *principal directions* of the data.
- ▶ PCA generates uncorrelated encodings equal to $Z_k = P_k^T X_0$, for some choice of k eigenvectors P_k .

P_k is also a solution to the following problem with orthonormality constraints, which is similar to the linear autoencoder ($I_{k \times k}$ is the $k \times k$ identity matrix):

$$\min_{W \in \mathbb{R}^{n \times k}} \|X_0 - WW^T X_0\|_F^2 \text{ s.t. } W^T W = I_{k \times k}$$

The solution P_k is not unique, $W = QP_k$ is also a solution for any orthogonal Q .

PCA and the minimum reconstruction error

Let λ_i denote the i th eigenvalue of Σ , and let w_i denote its associated eigenvector.

- ▶ The value of each λ_i corresponds to the variance of X explained in the direction of eigenvector w_i .
- ▶ As we include more principal components, λ_i 's, we explain more of the variation in X .

If $k \leq n$, the optimal reconstruction error of the single-layer linear autoencoder is bounded below by the sum of the eigenvalues beyond the k th:

$$\min_{W_1, W_2, b_1, b_2} \|X - \hat{X}\|_F^2 = \min_{W_1, W_2, b_1, b_2} \|X - W_2 (W_1 X + b_1 \ell_p^T) - b_2 \ell_p^T\|_F^2 = \sum_{i=k+1}^n \lambda_i$$

If Σ has rank k , we would have that $\lambda_{k+1} = \lambda_n = 0$ and the minimum reconstruction error would be zero.

If Σ has full rank, i.e., there are no perfect linear dependencies between any combination of features, the minimum reconstruction error would be zero only in the case where we chose $k = n$.

PCA and autoencoders

A linear autoencoder applies PCA to the input data by generating a projection of the data into a low-dimensional space. However:

- ▶ The encodings of the bottleneck are correlated and not sorted.
- ▶ The solutions to each choice of k are not nested, we need to train for each k .

When building and training an autoencoder, PCA serves as a useful benchmark.

- ▶ Autoencoders are more flexible, they may capture richer features through deep models and non-linearities.
- ▶ We can use loss functions that are not restricted to MSE.

Besides, through autoencoders, we can easily employ regularization techniques to avoid overfitting.

- ▶ PCA may detect noise as a latent encoding that is irrelevant for generalization out of sample.

Summary of Lesson 1

In Lesson 1, we have:

- ▶ Described the role and design of autoencoders.
- ▶ Represented autoencoders as neural networks.
- ▶ Described PCA and its relationship with autoencoders.

⇒ **References for this lesson:**

Chapter 7 in Ranjan, C. *Understanding Deep Learning*.

Plaut, Elad. "From Principal Subspaces to Principal Components with Linear Autoencoders." *arXiv*, 2018

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement an autoencoder to compress the information contained in stock prices.

See you there!